MODUL 08 · STATE MANAGEMENT

Manajemen State Terstruktur dengan Provider

Memahami batasan setState() dan bahaya 'prop drilling' pada aplikasi skala besar. Menguasai Provider: ChangeNotifier, ChangeNotifierProvider, context.watch/read/select, Consumer, Selector, MultiProvider, pola state async (isLoading/error/data), ProxyProvider, struktur folder model/service/provider/UI, auth gate, serta kesalahan umum dan cara memperbaikinya.

Durasi Pertemuan 8 (150 Menit) + Belajar Mandiri

Prasyarat Modul 07: Navigasi & Routing

Capaian Pembelajaran

- Menganalisis perbedaan Ephemeral State (State Lokal) vs App State (State Global).
- Menjelaskan alur reaktif ChangeNotifier dan notifyListeners().
- Menerapkan Provider untuk memisahkan Business Logic dari kode antarmuka UI.
- Mengoptimalkan performa render dengan context.select, Consumer, dan Selector.
- Mengelola beberapa provider dengan MultiProvider dan ChangeNotifierProxyProvider.
- Menerapkan pola state async isLoading/error/data untuk data dari service.
- Menyusun struktur folder model/service/provider/UI dan auth gate berbasis Provider.
- Mengenali dan memperbaiki kesalahan umum: read di build, provider di initState, notify setelah dispose.

Daftar Isi Modul

1. Masalah 'Prop Drilling' & Kapan Butuh State Management?
2. ChangeNotifier dari Nol
3. ChangeNotifierProvider: Menyediakan State ke Pohon Widget
4. watch, read, select, Consumer & Selector
5. Tiga Pilar Utama Arsitektur Provider: Contoh KRS
6. MultiProvider
7. State Async: isLoading, error, data
8. ProxyProvider (Pengenalan)
9. Struktur Folder & Auth Gate
10. Kesalahan Umum & Cara Memperbaikinya
11. Studi Kasus: Aplikasi Tugas Kuliah Multi-Halaman
12. Tugas Praktikum
13. Kuis Pemahaman & Kunci Jawaban

1. Masalah 'Prop Drilling' & Kapan Butuh State Management?

Ketika aplikasi Flutter tumbuh besar dan memiliki puluhan halaman, mengandalkan `setState()` saja akan menimbulkan masalah serius yang disebut **Prop Drilling**.

Dua jenis state

Jenis	Contoh	Alat yang cocok
Ephemeral state (state lokal)	Tab aktif, isi TextField, checkbox "ingat saya", animasi	`setState` di `StatefulWidget`
App state (state bersama)	User yang login, isi keranjang/KRS, daftar todo, tema gelap	State management, misalnya **Provider**

Aturan praktis: jika data hanya dipakai satu widget, `setState` sudah cukup. Jika data dipakai oleh beberapa widget yang berjauhan atau beberapa halaman, saatnya memakai Provider.

Masalah Prop Drilling vs Solusi Provider

❌ **Tanpa State Management (Prop Drilling)**

Jika Widget Anak di tingkat ke-5 membutuhkan data Mahasiswa yang ada di Root, data tersebut harus dioper satu per satu melewati Widget Tingkat 1 → 2 → 3 → 4 → 5 lewat konstruktor! Widget perantara terpaksa memegang data yang tidak mereka gunakan.

✅ **Dengan Provider (Dependency Injection)**

Data state diletakkan di atas pohon aplikasi (Root/Scope). Widget manapun di kedalaman berapa pun dapat langsung mengakses data dan mendengarkan perubahannya hanya dengan `context.watch()` tanpa melalui widget perantara!

Contoh kecil: data dititipkan dari level ke level

```
// Level 1 punya data
class HomePage extends StatelessWidget {
  final int jumlahKeranjang;
  const HomePage({super.key, required this.jumlahKeranjang});

  @override
  Widget build(BuildContext context) => Header(jumlahKeranjang: jumlahKeranjang);
}
```

```dart
// Level 2 TIDAK memakai data, hanya meneruskan
class Header extends StatelessWidget {
  final int jumlahKeranjang;
  const Header({super.key, required this.jumlahKeranjang});

  @override
  Widget build(BuildContext context) => IkonKeranjang(jumlah: jumlahKeranjang);
}

// Level 3 yang benar-benar butuh data
class IkonKeranjang extends StatelessWidget {
  final int jumlah;
  const IkonKeranjang({super.key, required this.jumlah});

  @override
  Widget build(BuildContext context) => Text('🛒 $jumlah');
}
```

`Header` sama sekali tidak memakai `jumlahKeranjang`, tetapi tetap wajib menerimanya hanya untuk diteruskan ke `IkonKeranjang`.

Contoh lengkap: katalog dan keranjang tanpa Provider

Jalankan kode ini dan hitung berapa constructor yang menerima data "titipan".

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Contoh TANPA Provider: perhatikan berapa banyak data yang "dititipkan"
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KatalogPage(),
    );
  }
}

// Pemilik state: semua data keranjang ada di sini
class KatalogPage extends StatefulWidget {
  const KatalogPage({super.key});

  @override
  State<KatalogPage> createState() => _KatalogPageState();
}
```

```dart
class _KatalogPageState extends State<KatalogPage> {
  final List<String> _keranjang = [];

  void _tambah(String matkul) => setState(() => _keranjang.add(matkul));

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Titipan 1: jumlah dioper ke AppBarKatalog
      appBar: AppBarKatalog(jumlah: _keranjang.length, isi: _keranjang),
      // Titipan 2: callback dioper ke DaftarMatkul -> ItemMatkul
      body: DaftarMatkul(onTambah: _tambah),
    );
  }
}

class AppBarKatalog extends StatelessWidget implements PreferredSizeWidget {
  final int jumlah;
  final List<String> isi;

  const AppBarKatalog({super.key, required this.jumlah, required this.isi});

  @override
  Size get preferredSize => const Size.fromHeight(kToolbarHeight);

  @override
  Widget build(BuildContext context) {
    return AppBar(
      title: const Text('Katalog (tanpa Provider)'),
      // Titipan 3: data dioper lagi ke TombolKeranjang
      actions: [TombolKeranjang(jumlah: jumlah, isi: isi)],
    );
  }
}

class TombolKeranjang extends StatelessWidget {
  final int jumlah;
  final List<String> isi;

  const TombolKeranjang({super.key, required this.jumlah, required this.isi});

  @override
  Widget build(BuildContext context) {
    return IconButton(
      tooltip: 'Keranjang',
      icon: Badge(label: Text('$jumlah'), child: const Icon(Icons.shopping_cart)),
      onPressed: () {
        // Titipan 4: data dioper ke halaman lain lewat constructor
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => KeranjangPage(isi: isi)),
        );
      },
    );
  }
}

class DaftarMatkul extends StatelessWidget {
  final ValueChanged<String> onTambah; // tidak dipakai di sini, hanya diteruskan
```

```dart
  const DaftarMatkul({super.key, required this.onTambah});

  static const matkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        for (final m in matkul) ItemMatkul(nama: m, onTambah: onTambah),
      ],
    );
  }
}

class ItemMatkul extends StatelessWidget {
  final String nama;
  final ValueChanged<String> onTambah;

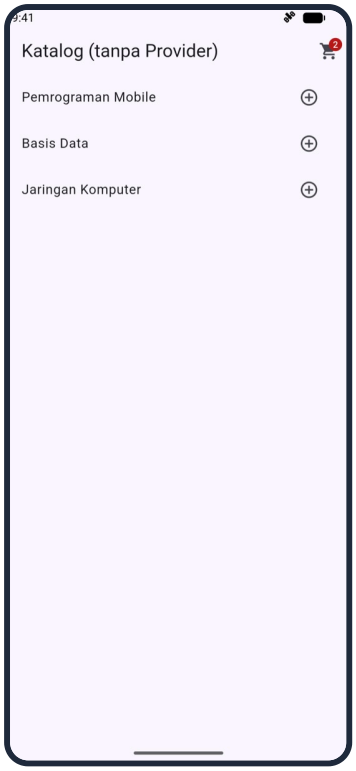
  const ItemMatkul({super.key, required this.nama, required this.onTambah});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(nama),
      trailing: IconButton(
        tooltip: 'Tambah $nama',
        icon: const Icon(Icons.add_circle_outline),
        onPressed: () => onTambah(nama), // akhirnya dipakai di level terdalam
      ),
    );
  }
}

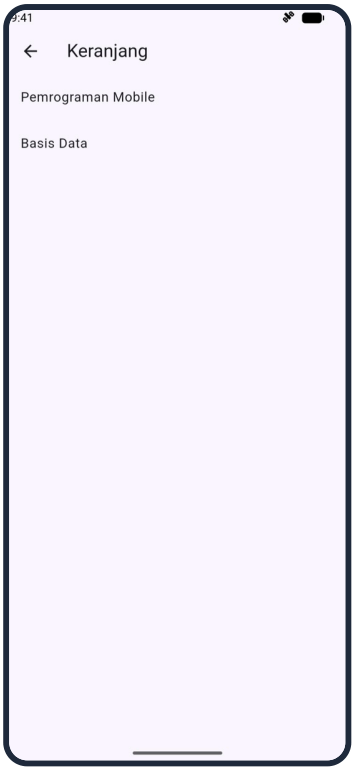
class KeranjangPage extends StatelessWidget {
  final List<String> isi;

  const KeranjangPage({super.key, required this.isi});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Keranjang')),
      // Masalah tambahan: halaman ini tidak bisa MENGHAPUS item
      // tanpa menitipkan callback lagi dari KatalogPage.
      body: ListView(children: [for (final m in isi) ListTile(title: Text(m))]),
    );
  }
}
```



Katalog: badge menerima jumlah lewat constructor



Keranjang menerima isi lewat constructor

Analisis masalah:

- `AppBarKatalog` dan `DaftarMatkul` hanya menjadi "kurir": menerima data/callback lalu meneruskannya.
- Menambah fitur hapus di `KeranjangPage` berarti menambah callback baru yang harus melewati beberapa widget lagi.
- `KeranjangPage` menerima *salinan referensi* list saat dibuka. Jika list berubah, halaman itu tidak tahu harus rebuild karena tidak ada yang memanggil `setState` di sana.
- Setiap `setState` di `KatalogPage` membangun ulang seluruh halaman, termasuk bagian yang tidak berubah.

> ℹ️ **Apa yang akan kita pelajari**
>
> Di section berikutnya kita membangun solusinya selangkah demi selangkah: `ChangeNotifier` (tempat state), `ChangeNotifierProvider` (cara menyediakan state ke pohon widget), lalu `watch` / `read` / `select` (cara membaca state dari widget mana pun).

⚠️ **Kesalahan umum**

- Langsung memakai Provider untuk *semua* state. Isi TextField atau tab aktif tetap lebih sederhana dengan `setState`.
- Menyimpan state bersama di variabel global biasa. UI tidak akan otomatis rebuild ketika nilainya berubah.

2. ChangeNotifier dari Nol

`ChangeNotifier` adalah class bawaan Flutter (bukan dari package provider) yang bisa "mengabari" siapa saja yang mendengarkan ketika datanya berubah. Inilah pondasi Provider.

Empat langkah membuat ChangeNotifier

1. Buat class yang `extends ChangeNotifier`.
2. Simpan state di field **privat** (diawali `_`).
3. Sediakan **getter** untuk membaca state.
4. Sediakan **method** untuk mengubah state, dan panggil `notifyListeners()` di akhir.

Contoh kecil: tanpa UI sama sekali

```dart
import 'package:flutter/foundation.dart';

class CounterModel extends ChangeNotifier {
  int _nilai = 0; // 1. state disimpan privat

  int get nilai => _nilai; // 2. dibaca lewat getter

  void tambah() {
    _nilai++; // 3. ubah state
    notifyListeners(); // 4. kabari semua pendengar
  }
}

void main() {
  final counter = CounterModel();

  // Daftarkan "pendengar": fungsi yang dipanggil setiap notifyListeners()
  counter.addListener(() {
    debugPrint('Nilai berubah menjadi ${counter.nilai}');
  });

  counter.tambah(); // Nilai berubah menjadi 1
  counter.tambah(); // Nilai berubah menjadi 2
}
```

- `addListener` mendaftarkan fungsi yang akan dipanggil setiap `notifyListeners()`.
- Dari luar class, `_nilai` tidak bisa diubah langsung. Satu-satunya jalan adalah lewat `tambah()`. Ini menjaga aturan bisnis tetap di satu tempat.

Contoh lengkap: menghubungkan ke UI dengan ListenableBuilder

Sebelum memakai package provider, mari lihat bahwa ChangeNotifier sudah bisa dipakai dengan widget bawaan `ListenableBuilder`.

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// ChangeNotifier TANPA package provider, memakai ListenableBuilder bawaan Flutter
import 'package:flutter/material.dart';

class CounterModel extends ChangeNotifier {
  int _nilai = 0;
  int get nilai => _nilai;

  void tambah() {
    _nilai++;
    notifyListeners();
  }

  void reset() {
    _nilai = 0;
    notifyListeners();
  }
}

// Satu objek model untuk seluruh aplikasi (sementara, sebelum pakai Provider)
final counter = CounterModel();

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const CounterPage(),
    );
  }
}

// Perhatikan: StatelessWidget, tidak ada setState sama sekali
class CounterPage extends StatelessWidget {
  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('ChangeNotifier dari Nol')),
      body: Center(
        // ListenableBuilder = addListener + rebuild otomatis
        child: ListenableBuilder(
          listenable: counter,
          builder: (context, child) {
            return Text(
              'Jumlah klik: ${counter.nilai}',
              style: Theme.of(context).textTheme.headlineMedium,
            );
          },
```

```
        ),
      ),
      floatingActionButton: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          FloatingActionButton(
            heroTag: 'reset',
            tooltip: 'Reset',
            onPressed: counter.reset,
            child: const Icon(Icons.refresh),
          ),
          const SizedBox(width: 12),
          FloatingActionButton(
            heroTag: 'tambah',
            tooltip: 'Tambah',
            onPressed: counter.tambah,
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```



Tiga kali Tambah tanpa setState

Penjelasan:

- `CounterPage` adalah `StatelessWidget`, tetapi angkanya tetap berubah. Yang memicu rebuild adalah `ListenableBuilder`, bukan `setState`.
- Hanya bagian di dalam `builder` yang di-build ulang. `AppBar` dan tombol tidak ikut.

- Kelemahan contoh ini: `counter` adalah variabel global. Sulit diganti saat testing, dan tidak pernah di-`dispose`. Package provider menyelesaikan kedua masalah ini.

> ⚠️ **Kesalahan umum**
>
> - **Lupa `notifyListeners()`.** Data berubah di memori, tetapi layar tidak berubah sama sekali.
> - Memanggil `notifyListeners()` di dalam getter. Getter hanya membaca; memanggil notify di sana menyebabkan rebuild tanpa henti.
> - Membuat field publik (`int nilai = 0;`) lalu mengubahnya dari UI: `model.nilai++`. Tidak ada yang memanggil notify, dan aturan bisnis tersebar di UI.

3. ChangeNotifierProvider: Menyediakan State ke Pohon Widget

`ChangeNotifierProvider` dari package **provider** "menitipkan" sebuah ChangeNotifier di pohon widget. Semua widget di bawahnya bisa mengambil objek itu lewat `context`, tanpa constructor.

Instalasi

```
flutter pub add provider
```

Contoh kecil: mendaftarkan provider

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class CounterModel extends ChangeNotifier {
  int _nilai = 0;
  int get nilai => _nilai;

  void tambah() {
    _nilai++;
    notifyListeners();
  }
}

void main() {
  runApp(
    // create dipanggil SEKALI; Provider juga otomatis memanggil dispose()
    ChangeNotifierProvider(
      create: (context) => CounterModel(),
      child: const MyApp(), // seluruh MyApp & turunannya bisa mengakses CounterModel
    ),
  );
}
```

- `create` dipanggil sekali (secara *lazy*, saat pertama kali dibaca) untuk membuat objek.
- Saat provider dibuang dari pohon widget, ia otomatis memanggil `dispose()` pada objek tersebut.
- Provider di atas `MaterialApp` berarti tersedia di **semua halaman**, termasuk halaman yang dibuka dengan `Navigator.push`.

Contoh lengkap: satu state, dua halaman

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class CounterModel extends ChangeNotifier {
```

```
  int _nilai = 0;
  int get nilai => _nilai;

  void tambah() {
    _nilai++;
    notifyListeners();
  }

  void kurang() {
    if (_nilai == 0) return;
    _nilai--;
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const HalamanSatu(),
    );
  }
}

class HalamanSatu extends StatelessWidget {
  const HalamanSatu({super.key});

  @override
  Widget build(BuildContext context) {
    // watch: widget ini rebuild setiap CounterModel memanggil notifyListeners
    final nilai = context.watch<CounterModel>().nilai;

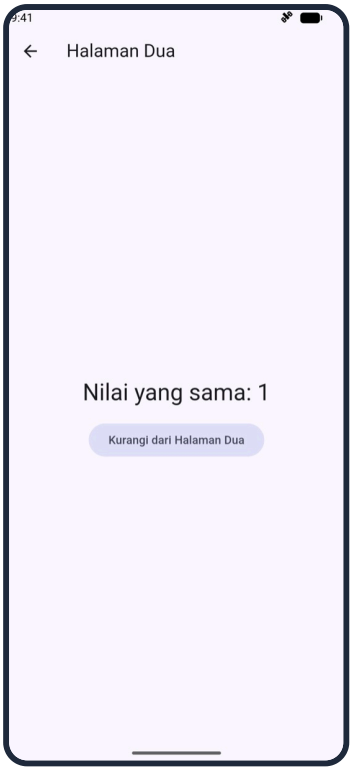
    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Satu')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Nilai: $nilai', style: Theme.of(context).textTheme.headlineMedium),
            const SizedBox(height: 16),
            FilledButton(
              // read: hanya memanggil method, tidak berlangganan
              onPressed: () => context.read<CounterModel>().tambah(),
              child: const Text('Tambah'),
            ),
            const SizedBox(height: 8),
```

```
            OutlinedButton(
              // Tidak ada data yang dikirim lewat constructor!
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute(builder: (_) => const HalamanDua()),
              ),
              child: const Text('Buka Halaman Dua'),
            ),
          ],
        ),
      ),
    );
  }
}

class HalamanDua extends StatelessWidget {
  const HalamanDua({super.key});

  @override
  Widget build(BuildContext context) {
    final nilai = context.watch<CounterModel>().nilai;

    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Dua')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Nilai yang sama: $nilai',
                style: Theme.of(context).textTheme.headlineMedium),
            const SizedBox(height: 16),
            FilledButton.tonal(
              onPressed: () => context.read<CounterModel>().kurang(),
              child: const Text('Kurangi dari Halaman Dua'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Halaman Dua membaca dan mengubah state yang sama

Penjelasan:

- `HalamanDua` dibuka tanpa mengirim data apa pun, tetapi bisa membaca dan mengubah nilai yang sama.
- Saat menekan "Kurangi" di Halaman Dua lalu kembali, Halaman Satu sudah menampilkan nilai terbaru karena ia juga `watch` objek yang sama.
- `context.watch<CounterModel>()` dipakai di `build`; `context.read<CounterModel>()` dipakai di `onPressed`. Bedanya dibahas di section berikutnya.

> ⚠️ **Kesalahan umum**
>
> - **ProviderNotFoundException.** Muncul jika widget mencari provider yang tidak ada di atasnya. Penyebab tersering: provider dipasang di dalam satu halaman, lalu halaman lain (hasil `push`) mencoba membacanya. Pasang provider di atas `MaterialApp` jika dipakai lintas halaman.
> - Lupa menulis tipe generik: `context.watch()` tanpa `<CounterModel>` membuat Dart tidak tahu provider mana yang dicari.
> - Memakai `Provider` biasa (bukan `ChangeNotifierProvider`) untuk ChangeNotifier. Hasilnya UI tidak pernah rebuild.

4. watch, read, select, Consumer & Selector

Ada beberapa cara membaca provider. Perbedaannya ada pada **kapan widget di-build ulang**. Memilih cara yang tepat membuat aplikasi benar sekaligus hemat rebuild.

Ringkasan lima cara

```dart
// 1) watch: baca + langganan SEMUA perubahan (di dalam build)
final profil = context.watch<ProfilProvider>();

// 2) select: baca + langganan SATU nilai saja (di dalam build)
final nama = context.select((ProfilProvider p) => p.nama);

// 3) read: ambil objek tanpa langganan (di callback/event)
void onTekan() => context.read<ProfilProvider>().tambahPoin();

// 4) Consumer: langganan, tetapi hanya bagian builder yang rebuild
final widgetPoin = Consumer<ProfilProvider>(
  builder: (context, p, child) => Text('Poin: ${p.poin}'),
);

// 5) Selector: seperti Consumer + select
final widgetPoin2 = Selector<ProfilProvider, int>(
  selector: (context, p) => p.poin,
  builder: (context, poin, child) => Text('Poin: $poin'),
);
```

Cara	Rebuild ketika	Dipakai di
`context.watch<T>()`	Setiap `notifyListeners()`	`build()`
`context.select<T, R>(fn)`	Hanya jika nilai hasil `fn` berubah	`build()`
`context.read<T>()`	Tidak pernah	Callback: `onPressed`, `onTap`, `initState` (lewat post-frame)
`Consumer<T>`	Setiap notify, tetapi hanya isi `builder`	Di tengah pohon widget
`Selector<T, R>`	Hanya jika nilai `selector` berubah, hanya isi `builder`	Di tengah pohon widget

Contoh lengkap: membuktikan perbedaan rebuild

Setiap kartu menampilkan berapa kali ia di-build. Tekan tombol dan amati angka mana yang naik.

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Demo rebuild: angka "build ke-N" menunjukkan berapa kali tiap kartu di-build
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class ProfilProvider extends ChangeNotifier {
  String _nama = 'Andi';
  int _poin = 0;

  String get nama => _nama;
  int get poin => _poin;

  void tambahPoin() {
    _poin++;
    notifyListeners();
  }

  void gantiNama() {
    _nama = _nama == 'Andi' ? 'Bunga' : 'Andi';
    notifyListeners();
  }
}

// Penghitung build HANYA untuk keperluan demo (jangan ditiru di aplikasi nyata)
final Map<String, int> jumlahBuild = {};
int catatBuild(String nama) => jumlahBuild[nama] = (jumlahBuild[nama] ?? 0) + 1;

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => ProfilProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      home: const DemoPage(),
    );
  }
}

class DemoPage extends StatelessWidget {
  const DemoPage({super.key});

  @override
  Widget build(BuildContext context) {
    // Halaman ini TIDAK watch apa pun, jadi tidak ikut rebuild
    return Scaffold(
      appBar: AppBar(title: const Text('watch vs select vs Consumer')),
```

```
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          const KartuWatch(),
          const KartuSelectNama(),
          const KartuConsumer(),
          const KartuSelectorPoin(),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: FilledButton(
                  onPressed: () => context.read<ProfilProvider>().tambahPoin(),
                  child: const Text('Tambah Poin'),
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: OutlinedButton(
                  onPressed: () => context.read<ProfilProvider>().gantiNama(),
                  child: const Text('Ganti Nama'),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

// 1) watch: rebuild untuk SETIAP perubahan (nama maupun poin)
class KartuWatch extends StatelessWidget {
  const KartuWatch({super.key});

  @override
  Widget build(BuildContext context) {
    final p = context.watch<ProfilProvider>();
    final n = catatBuild('watch');
    return Card(
      child: ListTile(
        title: Text('watch: ${p.nama}, ${p.poin} poin'),
        subtitle: Text('build ke-$n'),
      ),
    );
  }
}

// 2) select: rebuild HANYA jika nama berubah
class KartuSelectNama extends StatelessWidget {
  const KartuSelectNama({super.key});

  @override
  Widget build(BuildContext context) {
    final nama = context.select((ProfilProvider p) => p.nama);
    final n = catatBuild('select');
    return Card(
      child: ListTile(
        title: Text('select(nama): $nama'),
```

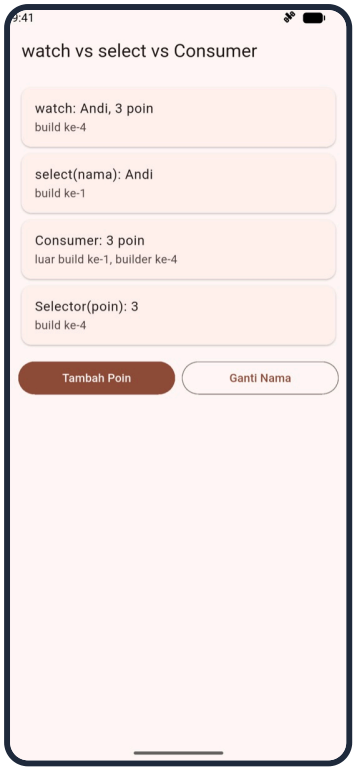
```
        subtitle: Text('build ke-$n'),
      ),
    );
  }
}

// 3) Consumer: kartu luar tidak rebuild, hanya isi builder
class KartuConsumer extends StatelessWidget {
  const KartuConsumer({super.key});

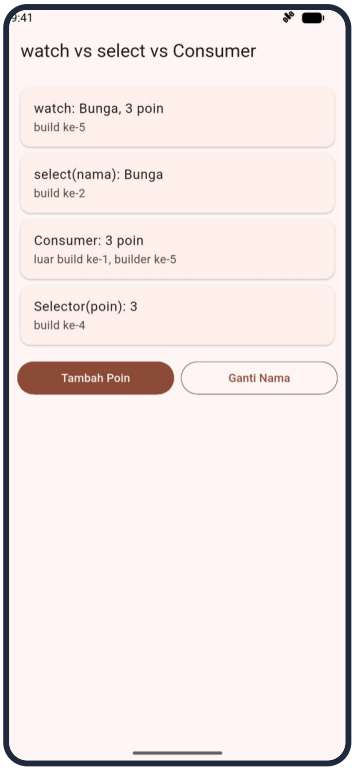
  @override
  Widget build(BuildContext context) {
    final nLuar = catatBuild('consumer-luar');
    return Card(
      child: Consumer<ProfilProvider>(
        builder: (context, p, child) {
          final nDalam = catatBuild('consumer-dalam');
          return ListTile(
            title: Text('Consumer: ${p.poin} poin'),
            subtitle: Text('luar build ke-$nLuar, builder ke-$nDalam'),
          );
        },
      ),
    );
  }
}

// 4) Selector: builder rebuild HANYA jika poin berubah
class KartuSelectorPoin extends StatelessWidget {
  const KartuSelectorPoin({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Selector<ProfilProvider, int>(
        selector: (context, p) => p.poin,
        builder: (context, poin, child) {
          final n = catatBuild('selector');
          return ListTile(
            title: Text('Selector(poin): $poin'),
            subtitle: Text('build ke-$n'),
          );
        },
      ),
    );
  }
}
```



Setelah 3x Tambah Poin: kartu select(nama) tidak rebuild



Setelah Ganti Nama: Selector(poin) tidak rebuild

Penjelasan hasil:

- **Tambah Poin**: kartu *watch*, *Consumer (builder)*, dan *Selector* naik. Kartu *select(nama)* tetap karena nama tidak berubah.
- **Ganti Nama**: kartu *watch*, *select(nama)*, dan *Consumer (builder)* naik. *Selector(poin)* tetap.
- Angka "luar" pada kartu Consumer tidak pernah naik. Hanya bagian di dalam `builder` yang di-build ulang.
- `DemoPage` sendiri tidak pernah rebuild karena hanya memakai `read` di tombol.

> 💡 **Cara memilih**
>
> 1. Di callback? Pakai `read`.
> 2. Di `build` dan butuh banyak field? Pakai `watch`.
> 3. Di `build` dan butuh satu nilai saja? Pakai `select`.
> 4. Widget besar tapi hanya sebagian kecil yang berubah? Bungkus bagian kecil itu dengan `Consumer` / `Selector`, atau pecah menjadi widget sendiri.

⚠️ **Kesalahan umum**

- `context.read` di dalam `build`: layar tidak ter-update karena widget tidak berlangganan.
- `context.watch` di dalam `onPressed`: error saat tombol ditekan.
- `select` yang mengembalikan list baru setiap kali, misalnya `select((p) => p.items.where(...).toList())`. List baru selalu dianggap "berbeda", sehingga select tidak menghemat apa pun. Pilih nilai sederhana seperti `int`, `bool`, atau `String`.

5. Tiga Pilar Utama Arsitektur Provider: Contoh KRS

Sekarang kita gabungkan tiga pilar Provider dalam satu contoh nyata: pengisian KRS dengan batas maksimal 24 SKS.

Untuk mengimplementasikan package `provider` (dari pub.dev), kita menggunakan 3 pilar utama:

1. **ChangeNotifier**: tempat state dan logika bisnis.
2. **ChangeNotifierProvider**: menyediakan state ke pohon widget.
3. **watch / read**: membaca state dari widget.

Pasang package terlebih dahulu: `flutter pub add provider`.

Langkah 1: Membuat Model State dengan ChangeNotifier

Buat folder `lib/providers/` lalu file `krs_provider.dart`:

`lib/providers/krs_provider.dart`

```dart
// lib/providers/krs_provider.dart
import 'package:flutter/foundation.dart';

class MataKuliah {
  final String nama;
  final int sks;

  const MataKuliah(this.nama, this.sks);
}

class KrsProvider with ChangeNotifier {
  // Data privat di dalam store
  final List<MataKuliah> _matkulTerpilih = [];
  final int batasMaksimalSks = 24;

  // Getter publik (read-only agar tidak bisa diubah langsung dari luar)
  List<MataKuliah> get matkulTerpilih => List.unmodifiable(_matkulTerpilih);
  int get totalSks => _matkulTerpilih.fold(0, (total, mk) => total + mk.sks);

  // Logika bisnis untuk menambah mata kuliah
  bool tambahMatkul(MataKuliah matkul) {
    if (totalSks + matkul.sks > batasMaksimalSks) {
      return false; // Melebihi batas kuota SKS!
    }

    _matkulTerpilih.add(matkul);

    // SANGAT PENTING: Beritahu seluruh widget yang mendengarkan agar me-rebuild UI!
    notifyListeners();
    return true;
  }

  void hapusMatkul(int index) {
    _matkulTerpilih.removeAt(index);
    notifyListeners();
```

```
  }
}
```

Perhatikan bahwa `tambahMatkul` mengembalikan `bool`. Provider yang memutuskan boleh/tidaknya (aturan bisnis), sedangkan UI hanya menampilkan pesan.

Langkah 2: Mendaftarkan Provider di Root Aplikasi (main.dart)

```dart
// Potongan inti dari lib/main.dart (kode lengkap ada di Langkah 3)
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/krs_provider.dart';

void main() {
  runApp(
    // Membungkus aplikasi dengan ChangeNotifierProvider
    ChangeNotifierProvider(
      create: (context) => KrsProvider(),
      child: const MyApp(),
    ),
  );
}
```

Langkah 3: Mengonsumsi State di Widget (context.watch vs context.read)

Metode Akses	Tujuan & Sifat Reaktif	Tempat Penggunaan
`context.watch<T>()`	Membaca data DAN ikut me-render ulang widget setiap ada perubahan (`notifyListeners`).	Di dalam method `build()` saat menampilkan teks/angka.
`context.read<T>()`	Hanya membaca fungsi/method SATU KALI tanpa mendaftar untuk re-render.	Di dalam event handler seperti tombol `onPressed: () {}`.

Kode lengkap `lib/main.dart` yang siap dijalankan (katalog mata kuliah + ringkasan SKS):

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: (1) flutter pub add provider, (2) membuat lib/providers/krs_provider.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/krs_provider.dart';

void main() {
  runApp(
    // Membungkus aplikasi dengan ChangeNotifierProvider di root
    ChangeNotifierProvider(
      create: (context) => KrsProvider(),
```

```
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: KatalogKrsScreen(),
    );
  }
}

class KatalogKrsScreen extends StatelessWidget {
  const KatalogKrsScreen({super.key});

  static const List<MataKuliah> katalog = [
    MataKuliah("Pemrograman Mobile", 4),
    MataKuliah("Basis Data Lanjut", 3),
    MataKuliah("Kecerdasan Buatan", 3),
    MataKuliah("Jaringan Komputer", 3),
    MataKuliah("Rekayasa Perangkat Lunak", 3),
    MataKuliah("Keamanan Siber", 3),
    MataKuliah("Etika Profesi", 2),
    MataKuliah("Kerja Praktik", 4),
  ];

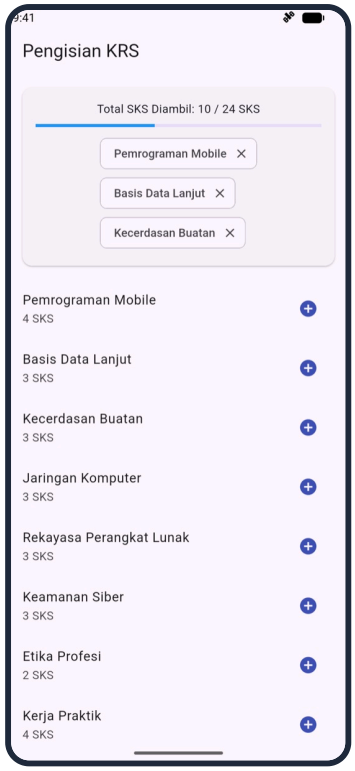
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Pengisian KRS")),
      body: Column(
        children: [
          const RingkasanKrsWidget(),
          Expanded(
            child: ListView.builder(
              itemCount: katalog.length,
              itemBuilder: (context, index) {
                final mk = katalog[index];
                return ListTile(
                  title: Text(mk.nama),
                  subtitle: Text("${mk.sks} SKS"),
                  trailing: IconButton(
                    tooltip: "Ambil ${mk.nama}",
                    icon: const Icon(Icons.add_circle, color: Colors.indigo),
                    onPressed: () {
                      // 2. Di event handler gunakan read: hanya memanggil method, tanpa rebuild
                      final berhasil = context.read<KrsProvider>().tambahMatkul(mk);
                      if (!berhasil) {
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(
                            content: Text("Kuota maksimal 24 SKS terlampaui!"),
                            backgroundColor: Colors.red,
                          ),
                        );
```

```
                    }
                  },
                ),
              );
            },
          ),
        ),
      ],
    ),
  );
  }
}

// Contoh Widget Tampilan KRS Menggunakan Provider
class RingkasanKrsWidget extends StatelessWidget {
  const RingkasanKrsWidget({super.key});

  @override
  Widget build(BuildContext context) {
    // 1. Menggunakan watch untuk mendengarkan perubahan angka SKS
    final krs = context.watch<KrsProvider>();

    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Text("Total SKS Diambil: ${krs.totalSks} / ${krs.batasMaksimalSks} SKS"),
            const SizedBox(height: 8),
            LinearProgressIndicator(
              value: krs.totalSks / krs.batasMaksimalSks,
              color: krs.totalSks >= 20 ? Colors.orange : Colors.blue,
            ),
            const SizedBox(height: 8),
            Wrap(
              spacing: 6,
              children: [
                for (var i = 0; i < krs.matkulTerpilih.length; i++)
                  InputChip(
                    label: Text(krs.matkulTerpilih[i].nama),
                    onDeleted: () => context.read<KrsProvider>().hapusMatkul(i),
                  ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```



Ringkasan SKS ikut berubah saat mata kuliah diambil

Penjelasan:

- `KatalogKrsScreen` tidak memakai `watch`, sehingga daftar katalog tidak ikut rebuild ketika SKS bertambah.
- `RingkasanKrsWidget` memakai `watch` sehingga total SKS, progress bar, dan chip selalu sinkron.
- `InputChip.onDeleted` memanggil `hapusMatkul` lewat `read`.
- `List.unmodifiable` di getter mencegah UI menambah item langsung tanpa melewati `tambahMatkul` (yang memeriksa kuota).

> ⚠️ **Kesalahan umum**
>
> - Memeriksa batas SKS di UI, bukan di provider. Jika ada dua tombol tambah di dua halaman, aturannya harus ditulis dua kali.
> - Mengembalikan `_matkulTerpilih` langsung dari getter. UI bisa memanggil `.add()` dan melewati pengecekan kuota.

6. MultiProvider

Aplikasi nyata biasanya punya lebih dari satu provider: sesi login, tema, keranjang, data mata kuliah, dan seterusnya. `MultiProvider` mendaftarkan semuanya sekaligus tanpa nesting yang menjorok ke kanan.

Tanpa vs dengan MultiProvider

```
// Tanpa MultiProvider (sulit dibaca)
ChangeNotifierProvider(
  create: ... TemaProvider,
  child: ChangeNotifierProvider(
    create: ... SesiProvider,
    child: ChangeNotifierProvider(
      create: ... FavoritProvider,
      child: MyApp(),
    ),
  ),
)
```

```
// Dengan MultiProvider
MultiProvider(
  providers: [
    ChangeNotifierProvider(create: ... Tema),
    ChangeNotifierProvider(create: ... Sesi),
    ChangeNotifierProvider(create: ... Favorit),
  ],
  child: MyApp(),
)
```

Contoh lengkap: tema, sesi, dan favorit

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

// Provider 1: pengaturan tampilan
class TemaProvider extends ChangeNotifier {
  bool _gelap = false;
  bool get gelap => _gelap;

  void toggle() {
    _gelap = !_gelap;
    notifyListeners();
  }
}

// Provider 2: data pengguna yang sedang login
class SesiProvider extends ChangeNotifier {
  String? _nama;
  String? get nama => _nama;
  bool get sudahLogin => _nama != null;

  void login(String nama) {
    _nama = nama;
    notifyListeners();
  }

  void logout() {
    _nama = null;
    notifyListeners();
  }
```

```dart
}

// Provider 3: daftar favorit
class FavoritProvider extends ChangeNotifier {
  final Set<String> _items = {};
  Set<String> get items => Set.unmodifiable(_items);
  bool isFavorit(String m) => _items.contains(m);

  void toggle(String m) {
    if (!_items.remove(m)) _items.add(m);
    notifyListeners();
  }
}

void main() {
  runApp(
    // MultiProvider = beberapa provider tanpa nesting bertingkat
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => TemaProvider()),
        ChangeNotifierProvider(create: (_) => SesiProvider()),
        ChangeNotifierProvider(create: (_) => FavoritProvider()),
      ],
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // MaterialApp hanya rebuild saat nilai "gelap" berubah
    final gelap = context.select((TemaProvider t) => t.gelap);
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      darkTheme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.dark),
      themeMode: gelap ? ThemeMode.dark : ThemeMode.light,
      home: const BerandaPage(),
    );
  }
}

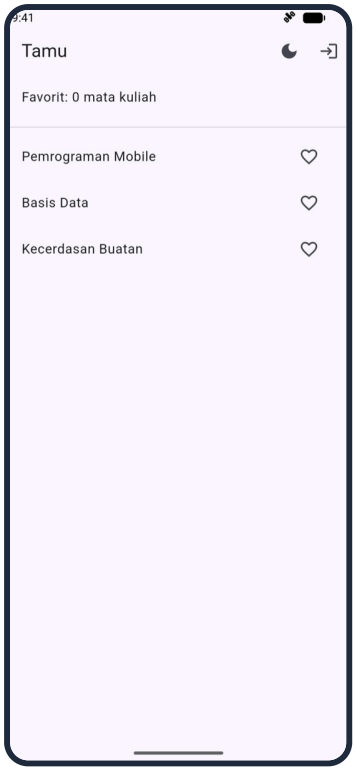
class BerandaPage extends StatelessWidget {
  const BerandaPage({super.key});

  static const matkul = ['Pemrograman Mobile', 'Basis Data', 'Kecerdasan Buatan'];

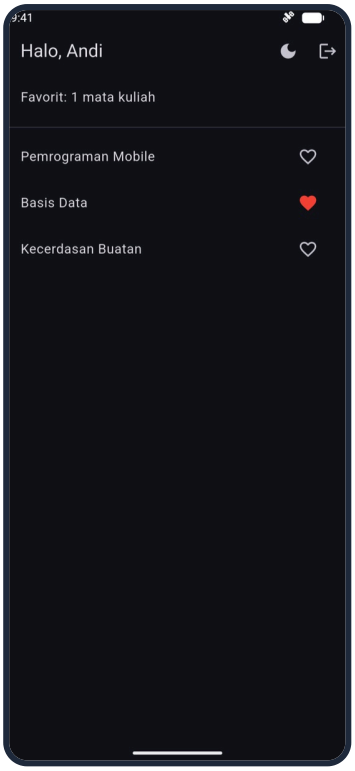
  @override
  Widget build(BuildContext context) {
    final sesi = context.watch<SesiProvider>();
    final favorit = context.watch<FavoritProvider>();

    return Scaffold(
      appBar: AppBar(
        title: Text(sesi.sudahLogin ? 'Halo, ${sesi.nama}' : 'Tamu'),
        actions: [
```

```
          IconButton(
            tooltip: 'Ganti tema',
            icon: const Icon(Icons.dark_mode),
            onPressed: () => context.read<TemaProvider>().toggle(),
          ),
          IconButton(
            tooltip: sesi.sudahLogin ? 'Logout' : 'Login',
            icon: Icon(sesi.sudahLogin ? Icons.logout : Icons.login),
            onPressed: () {
              final s = context.read<SesiProvider>();
              sesi.sudahLogin ? s.logout() : s.login('Andi');
            },
          ),
        ],
      ),
      body: ListView(
        children: [
          ListTile(title: Text('Favorit: ${favorit.items.length} mata kuliah')),
          const Divider(),
          for (final m in matkul)
            ListTile(
              title: Text(m),
              trailing: IconButton(
                tooltip: 'Favorit $m',
                icon: Icon(
                  favorit.isFavorit(m) ? Icons.favorite : Icons.favorite_border,
                  color: favorit.isFavorit(m) ? Colors.red : null,
                ),
                onPressed: () => context.read<FavoritProvider>().toggle(m),
              ),
            ),
        ],
      ),
    );
  }
}
```



Tiga provider: tema, sesi, favorit



Setelah login, menambah favorit, dan ganti tema

Penjelasan:

- Tiga provider dengan tanggung jawab berbeda. Masing-masing kecil dan mudah dipahami (prinsip *single responsibility*).
- `MyApp` memakai `context.select((TemaProvider t) => t.gelap)`. `MaterialApp` hanya rebuild ketika tema berganti, bukan ketika favorit bertambah.
- `BerandaPage` me-`watch` dua provider sekaligus. Halaman rebuild jika salah satunya berubah.
- Urutan di `providers: [...]` penting hanya jika satu provider bergantung pada provider lain (lihat ProxyProvider di section 8).

> ⚠️ **Kesalahan umum**
>
> - Membuat satu "GodProvider" raksasa berisi semua state. Setiap perubahan kecil membuat semua pendengar rebuild, dan file-nya sulit dirawat.
> - Mendaftarkan dua provider dengan tipe yang sama. Widget hanya bisa menemukan yang paling dekat; yang lain tertutup.

7. State Async: isLoading, error, data

Data dari internet atau database tidak datang seketika. Selama menunggu, UI harus menampilkan **loading**; jika gagal, menampilkan **error** dengan tombol coba lagi; jika berhasil, menampilkan **data**. Pola tiga serangkai `isLoading` / `error` / `data` ini akan dipakai ulang di Modul 09 (REST API) dan proyek CRUD Modul 13–14.

Pola di sisi UI

```dart
// Urutan pengecekan di UI: loading -> error -> kosong -> data
if (prov.isLoading) return const CircularProgressIndicator();
if (prov.error != null) return Text(prov.error!);
if (prov.data.isEmpty) return const Text('Belum ada data');
return ListView(children: [for (final d in prov.data) Text(d)]);
```

Pola di sisi provider

1. Set `_isLoading = true`, kosongkan `_error`, lalu `notifyListeners()`.
2. `try` : panggil service dan simpan hasilnya.
3. `catch` : simpan pesan error.
4. `finally` : set `_isLoading = false`, lalu `notifyListeners()`. Bagian `finally` selalu dijalankan, sehingga loading tidak "macet".

Contoh lengkap (multi-file)

```
lib/
├── main.dart
├── models/mahasiswa.dart
├── services/mahasiswa_service.dart
└── providers/mahasiswa_provider.dart
```

`lib/models/mahasiswa.dart`

```dart
// lib/models/mahasiswa.dart
class Mahasiswa {
  final String nim;
  final String nama;
  final double ipk;

  const Mahasiswa({required this.nim, required this.nama, required this.ipk});
}
```

`lib/services/mahasiswa_service.dart`

```dart
// lib/services/mahasiswa_service.dart
// Service = satu-satunya tempat yang tahu cara MENGAMBIL data.
// Saat ini datanya palsu; di Modul 09 isi fungsi ini diganti dengan HTTP request.
import '../models/mahasiswa.dart';

class MahasiswaService {
```

```
  Future<List<Mahasiswa>> ambilSemua({bool simulasiGagal = false}) async {
    await Future.delayed(const Duration(seconds: 3)); // pura-pura menunggu server

    if (simulasiGagal) {
      throw Exception('Server tidak dapat dihubungi');
    }

    return const [
      Mahasiswa(nim: 'H1D023001', nama: 'Andi Pratama', ipk: 3.45),
      Mahasiswa(nim: 'H1D023002', nama: 'Bunga Lestari', ipk: 3.81),
      Mahasiswa(nim: 'H1D023003', nama: 'Citra Ayu', ipk: 3.12),
      Mahasiswa(nim: 'H1D023004', nama: 'Dimas Saputra', ipk: 3.67),
    ];
  }
}
```

lib/providers/mahasiswa_provider.dart

```
// lib/providers/mahasiswa_provider.dart
import 'package:flutter/foundation.dart';

import '../models/mahasiswa.dart';
import '../services/mahasiswa_service.dart';

class MahasiswaProvider extends ChangeNotifier {
  MahasiswaProvider(this._service);

  final MahasiswaService _service;

  // Tiga serangkai state async
  bool _isLoading = false;
  String? _error;
  List<Mahasiswa> _data = [];

  bool get isLoading => _isLoading;
  String? get error => _error;
  List<Mahasiswa> get data => List.unmodifiable(_data);

  Future<void> muatData({bool simulasiGagal = false}) async {
    if (_isLoading) return; // cegah request ganda saat tombol ditekan berkali-kali

    _isLoading = true;
    _error = null;
    notifyListeners(); // UI menampilkan loading

    try {
      _data = await _service.ambilSemua(simulasiGagal: simulasiGagal);
    } catch (e) {
      _error = 'Gagal memuat data: $e';
    } finally {
      _isLoading = false;
      notifyListeners(); // UI menampilkan data ATAU error
    }
  }
}
```

lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/mahasiswa_provider.dart';
import 'services/mahasiswa_service.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      // ..muatData() langsung memulai pemuatan saat provider dibuat
      create: (_) => MahasiswaProvider(MahasiswaService())..muatData(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarMahasiswaPage(),
    );
  }
}

class DaftarMahasiswaPage extends StatelessWidget {
  const DaftarMahasiswaPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Data Mahasiswa'),
        actions: [
          IconButton(
            tooltip: 'Simulasi gagal',
            icon: const Icon(Icons.wifi_off),
            onPressed: () =>
                context.read<MahasiswaProvider>().muatData(simulasiGagal: true),
          ),
        ],
      ),
      body: const _IsiHalaman(),
    );
  }
}

class _IsiHalaman extends StatelessWidget {
  const _IsiHalaman();

  @override
  Widget build(BuildContext context) {
```

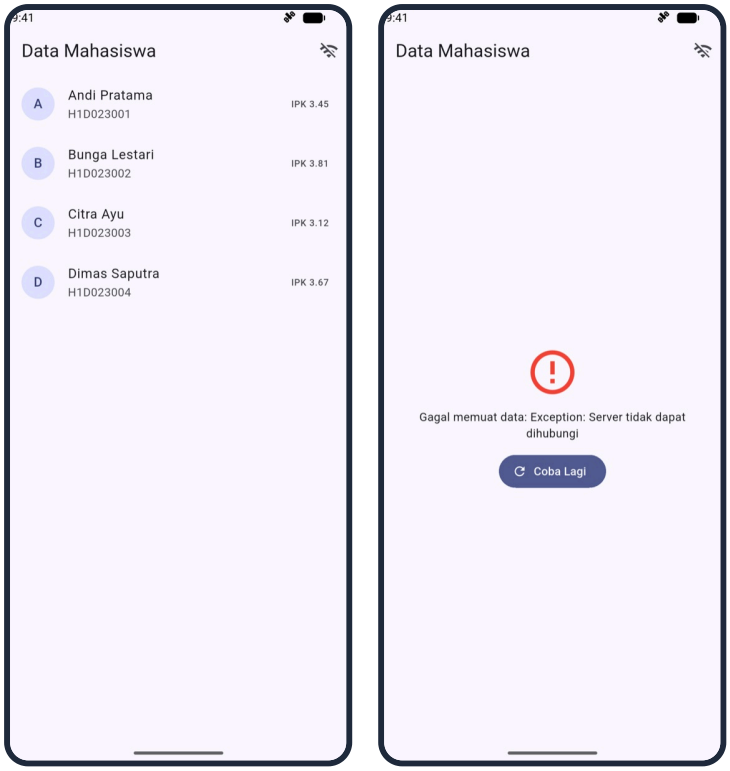
```dart
    final prov = context.watch<MahasiswaProvider>();

    // 1. Loading pertama kali (belum ada data sama sekali)
    if (prov.isLoading && prov.data.isEmpty) {
      return const Center(child: CircularProgressIndicator());
    }

    // 2. Error
    if (prov.error != null) {
      return Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Icon(Icons.error_outline, size: 64, color: Colors.red),
              const SizedBox(height: 12),
              Text(prov.error!, textAlign: TextAlign.center),
              const SizedBox(height: 12),
              FilledButton.icon(
                icon: const Icon(Icons.refresh),
                label: const Text('Coba Lagi'),
                onPressed: () => context.read<MahasiswaProvider>().muatData(),
              ),
            ],
          ),
        ),
      );
    }

    // 3. Data kosong
    if (prov.data.isEmpty) {
      return const Center(child: Text('Belum ada data'));
    }

    // 4. Data tersedia (+ tarik ke bawah untuk memuat ulang)
    return RefreshIndicator(
      onRefresh: () => context.read<MahasiswaProvider>().muatData(),
      child: ListView.builder(
        itemCount: prov.data.length,
        itemBuilder: (context, i) {
          final m = prov.data[i];
          return ListTile(
            leading: CircleAvatar(child: Text(m.nama[0])),
            title: Text(m.nama),
            subtitle: Text(m.nim),
            trailing: Text('IPK ${m.ipk.toStringAsFixed(2)}'),
          );
        },
      ),
    );
  }
}
```



State data

State error dengan tombol Coba Lagi

Penjelasan:

- `MahasiswaService` hanya tahu cara mengambil data. Ia tidak tahu apa pun soal UI atau `notifyListeners`.
- `MahasiswaProvider` menerima service lewat constructor (*dependency injection*). Saat testing (Modul 15), service ini bisa diganti dengan versi palsu.
- `create: (_) => MahasiswaProvider(MahasiswaService())..muatData()` : operator *cascade* `..` memanggil `muatData()` lalu tetap mengembalikan objek provider.
- `if (_isLoading) return;` mencegah request ganda saat tombol ditekan berkali-kali.
- Tombol ikon wifi mensimulasikan kegagalan sehingga tampilan error bisa dicoba tanpa mematikan internet.
- Kondisi loading ditulis `prov.isLoading && prov.data.isEmpty` : saat refresh (data lama masih ada), list tetap tampil dan indikator `RefreshIndicator` yang berputar.

> 💡 **Variasi: enum status**
>
> Sebagian tim memakai `enum Status { awal, loading, sukses, gagal }` sebagai pengganti beberapa boolean. Keduanya sah; yang penting UI selalu menangani semua kemungkinan.

⚠️ **Kesalahan umum**

- Lupa `notifyListeners()` sebelum `await`, sehingga loading tidak pernah terlihat.
- Tidak memakai `finally`: jika terjadi error, `isLoading` tetap `true` selamanya.
- Tidak mengosongkan `_error` saat mencoba lagi, sehingga pesan error lama tetap tampil walaupun request kedua berhasil.
- Memanggil service langsung dari widget (di `build`). Setiap rebuild akan memicu request baru.

8. ProxyProvider (Pengenalan)

Kadang satu provider butuh data dari provider lain. Contoh: daftar catatan bergantung pada siapa yang sedang login. `ChangeNotifierProxyProvider` menyambungkan keduanya: setiap kali provider sumber berubah, provider tujuan diberi tahu.

Bentuk umumnya

```
ChangeNotifierProxyProvider<Sumber, Tujuan>(
  create: (_) => Tujuan(),                                  // dibuat sekali
  update: (_, sumber, tujuan) => tujuan!..perbarui(sumber), // dipanggil tiap Sumber berubah
)
```

Contoh lengkap: catatan per pengguna

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class AuthProvider extends ChangeNotifier {
  String? _nim;
  String? get nim => _nim;

  void loginSebagai(String nim) {
    _nim = nim;
    notifyListeners();
  }
}

// CatatanProvider BERGANTUNG pada AuthProvider: catatan milik siapa?
class CatatanProvider extends ChangeNotifier {
  String? _nimAktif;
  final Map<String, List<String>> _catatanPerNim = {};

  List<String> get catatan => _catatanPerNim[_nimAktif] ?? const [];

  // Dipanggil oleh ProxyProvider setiap AuthProvider berubah
  void perbaruiPengguna(String? nim) {
    if (nim == _nimAktif) return;
    _nimAktif = nim;
    notifyListeners();
  }

  void tambah(String teks) {
    final nim = _nimAktif;
    if (nim == null) return;
    _catatanPerNim.putIfAbsent(nim, () => []).add(teks);
    notifyListeners();
  }
}

void main() {
```

```dart
  runApp(
    MultiProvider(
      providers: [
        // 1. Provider yang menjadi sumber harus didaftarkan LEBIH DULU
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        // 2. ProxyProvider: <Sumber, Hasil>
        ChangeNotifierProxyProvider<AuthProvider, CatatanProvider>(
          create: (_) => CatatanProvider(),
          // update dipanggil di awal dan setiap AuthProvider notifyListeners
          update: (_, auth, catatan) => catatan!..perbaruiPengguna(auth.nim),
        ),
      ],
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

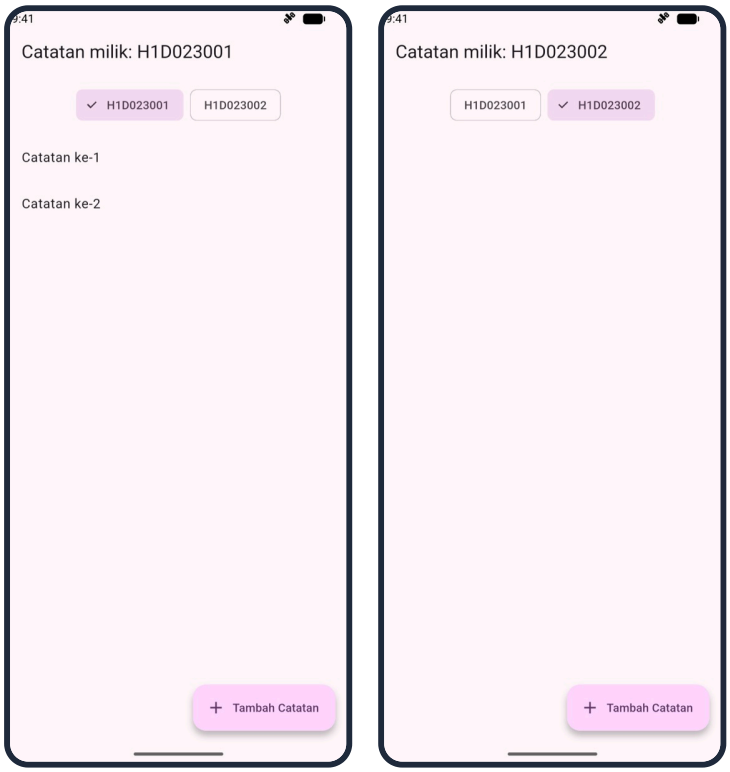
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.purple),
      home: const CatatanPage(),
    );
  }
}

class CatatanPage extends StatelessWidget {
  const CatatanPage({super.key});

  @override
  Widget build(BuildContext context) {
    final nim = context.watch<AuthProvider>().nim;
    final catatan = context.watch<CatatanProvider>().catatan;

    return Scaffold(
      appBar: AppBar(title: Text('Catatan milik: ${nim ?? '-'}')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Wrap(
              spacing: 8,
              children: [
                for (final n in ['H1D023001', 'H1D023002'])
                  ChoiceChip(
                    label: Text(n),
                    selected: nim == n,
                    onSelected: (_) => context.read<AuthProvider>().loginSebagai(n),
                  ),
              ],
            ),
          ),
          Expanded(
            child: ListView(
              children: [for (final c in catatan) ListTile(title: Text(c))],
```

```
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        icon: const Icon(Icons.add),
        label: const Text('Tambah Catatan'),
        onPressed: nim == null
            ? null
            : () {
                final prov = context.read<CatatanProvider>();
                prov.tambah('Catatan ke-${prov.catatan.length + 1}');
              },
      ),
    );
  }
}
```



Catatan milik H1D023001 | Ganti pengguna: CatatanProvider ikut diperbarui

Penjelasan:

- `AuthProvider` wajib didaftarkan **sebelum** ProxyProvider yang membutuhkannya.
- `update` menerima objek `CatatanProvider` yang sudah ada (`catatan!`) dan hanya memperbarui NIM-nya. Objek tidak dibuat ulang, jadi catatan yang tersimpan tidak hilang.
- Di `perbaruiPengguna`, `if (nim == _nimAktif) return;` mencegah notify yang tidak perlu.
- Pilih NIM pertama, tambah dua catatan, lalu pilih NIM kedua: daftar kosong. Kembali ke NIM pertama: dua catatan muncul lagi.

ℹ️ Seberapa sering dipakai?

Untuk proyek kuliah, ProxyProvider jarang wajib. Alternatif yang lebih sederhana: panggil `context.read<CatatanProvider>().perbaruiPengguna(nim)` tepat setelah login berhasil. Namun mengenal ProxyProvider membantu saat membaca kode proyek orang lain.

⚠️ Kesalahan umum

- Membuat objek baru di `update`: `update: (_, auth, __) => CatatanProvider()`. Semua data hilang setiap kali auth berubah.
- Urutan provider terbalik (ProxyProvider di atas AuthProvider) sehingga muncul ProviderNotFoundException.

9. Struktur Folder & Auth Gate

Semakin besar aplikasi, semakin penting memisahkan kode berdasarkan tanggung jawabnya. Struktur berikut sederhana, cukup untuk proyek kuliah, dan dipakai lagi di proyek CRUD + Auth (Modul 13–14).

```
lib/
├── main.dart              # runApp + MultiProvider + MaterialApp
├── models/                # class data murni (User, Mahasiswa, Todo)
├── services/              # akses data: HTTP, database, Supabase
├── providers/             # ChangeNotifier: state + logika bisnis
├── screens/               # halaman penuh (Scaffold)
└── widgets/               # komponen kecil yang dipakai ulang
```

Lapisan	Boleh mengimpor	Tidak boleh
models	Dart murni	Flutter widget, provider, service
services	models, package http/database	widget, `BuildContext`, provider
providers	models, services, `foundation.dart`	widget, `BuildContext`, `Navigator`
screens / widgets	models, providers, widget lain	Memanggil service langsung

Arah ketergantungan: **UI → Provider → Service → Model**. Provider tidak boleh tahu tentang UI; itulah sebabnya provider tidak menampilkan SnackBar atau melakukan navigasi sendiri.

Contoh lengkap: Auth gate dengan Provider

Auth gate adalah widget yang memilih menampilkan `LoginScreen` atau `HomeScreen` berdasarkan status sesi di provider. Login dan logout tidak memerlukan `Navigator.push` sama sekali.

`lib/models/user.dart`

```dart
// lib/models/user.dart
class User {
  final String email;
  final String nama;

  const User({required this.email, required this.nama});
}
```

`lib/services/auth_service.dart`

```dart
// lib/services/auth_service.dart
// Service palsu. Di Modul 12 bagian ini diganti dengan Supabase Auth.
import '../models/user.dart';
```

```dart
class AuthService {
  Future<User> login(String email, String password) async {
    await Future.delayed(const Duration(seconds: 1));
    if (password != '123456') {
      throw Exception('Email atau password salah');
    }
    return User(email: email, nama: email.split('@').first);
  }

  Future<void> logout() async {
    await Future.delayed(const Duration(milliseconds: 300));
  }
}
```

lib/providers/auth_provider.dart

```dart
// lib/providers/auth_provider.dart
import 'package:flutter/foundation.dart';

import '../models/user.dart';
import '../services/auth_service.dart';

class AuthProvider extends ChangeNotifier {
  AuthProvider(this._service);

  final AuthService _service;

  User? _user;
  bool _isLoading = false;
  String? _error;

  User? get user => _user;
  bool get isLoggedIn => _user != null;
  bool get isLoading => _isLoading;
  String? get error => _error;

  Future<bool> login(String email, String password) async {
    _isLoading = true;
    _error = null;
    notifyListeners();
    try {
      _user = await _service.login(email, password);
      return true;
    } catch (e) {
      _error = e.toString().replaceFirst('Exception: ', '');
      return false;
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

  Future<void> logout() async {
    await _service.logout();
    _user = null;
    notifyListeners();
```

```
  }
}
```

lib/widgets/auth_gate.dart

```dart
// lib/widgets/auth_gate.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
import '../screens/home_screen.dart';
import '../screens/login_screen.dart';

// Auth gate: memilih halaman berdasarkan status sesi.
// Tidak ada Navigator.push sama sekali untuk login/logout!
class AuthGate extends StatelessWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    final sudahLogin = context.select((AuthProvider a) => a.isLoggedIn);
    return sudahLogin ? const HomeScreen() : const LoginScreen();
  }
}
```

lib/screens/login_screen.dart

```dart
// lib/screens/login_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _emailController = TextEditingController(text: 'andi@mhs.unsoed.ac.id');
  final _passwordController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    FocusScope.of(context).unfocus(); // tutup keyboard
    final berhasil = await context.read<AuthProvider>().login(
          _emailController.text.trim(),
          _passwordController.text,
        );
```

```
    // Jika berhasil, AuthGate otomatis menampilkan HomeScreen.
    if (!berhasil && mounted) {
      final pesan = context.read<AuthProvider>().error ?? 'Login gagal';
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(pesan)));
    }
  }

  @override
  Widget build(BuildContext context) {
    final isLoading = context.select((AuthProvider a) => a.isLoading);

    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            TextField(
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
            ),
            TextField(
              controller: _passwordController,
              obscureText: true,
              decoration: const InputDecoration(
                labelText: 'Password',
                helperText: 'Petunjuk: 123456',
              ),
            ),
            const SizedBox(height: 24),
            SizedBox(
              width: double.infinity,
              child: FilledButton(
                onPressed: isLoading ? null : _login,
                child: isLoading
                    ? const SizedBox(
                        width: 20,
                        height: 20,
                        child: CircularProgressIndicator(strokeWidth: 2),
                      )
                    : const Text('Masuk'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/screens/home_screen.dart

```
// lib/screens/home_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
```

```dart
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final user = context.watch<AuthProvider>().user;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            tooltip: 'Logout',
            icon: const Icon(Icons.logout),
            // Setelah logout, AuthGate otomatis kembali ke LoginScreen
            onPressed: () => context.read<AuthProvider>().logout(),
          ),
        ],
      ),
      body: Center(
        child: Text(
          'Selamat datang, ${user?.nama}!',
          style: Theme.of(context).textTheme.headlineSmall,
        ),
      ),
    );
  }
}
```

lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

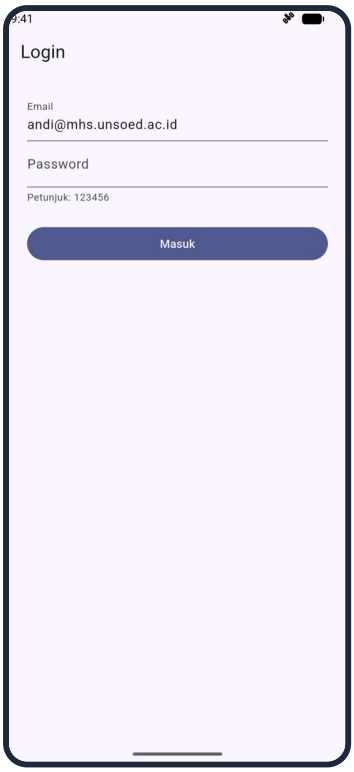
import 'providers/auth_provider.dart';
import 'services/auth_service.dart';
import 'widgets/auth_gate.dart';

void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider(AuthService())),
        // Provider lain (MatkulProvider, TodoProvider, ...) ditambahkan di sini
      ],
      child: const MyApp(),
    ),
  );
}

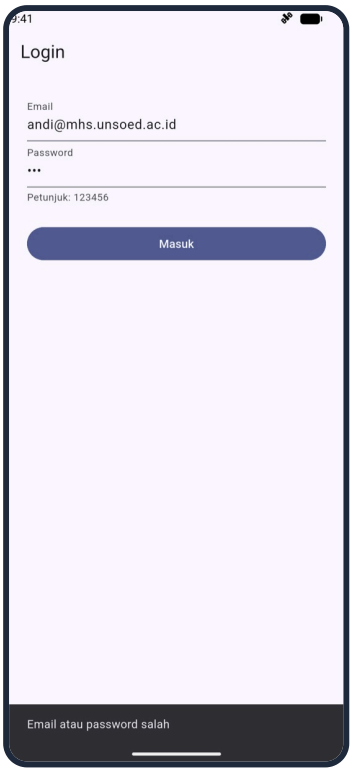
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

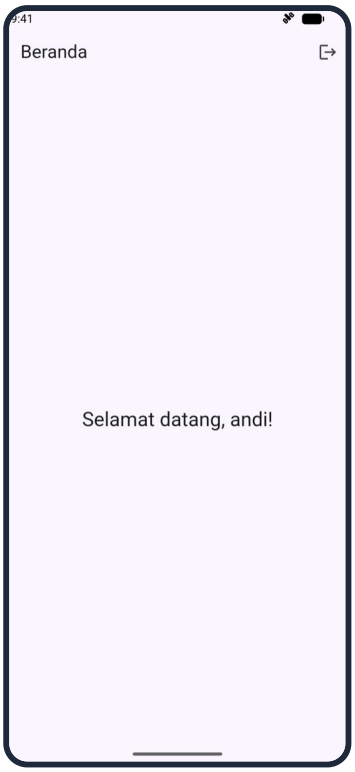
```
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const AuthGate(),
    );
  }
}
```



AuthGate menampilkan LoginScreen



Password salah: pesan error dari provider



Login berhasil: AuthGate menampilkan HomeScreen

Penjelasan:

- `AuthGate` memakai `select` pada `isLoggedIn`. Ia hanya rebuild saat status login berubah, bukan saat `isLoading` berubah.
- `AuthProvider.login` mengembalikan `bool`. Layar login yang memutuskan cara menampilkan error (SnackBar), sedangkan provider hanya menyimpan pesannya.
- Tombol Masuk dinonaktifkan dan diganti spinner selama `isLoading`, mencegah login ganda.
- Logout cukup memanggil `logout()`. Begitu `_user` menjadi `null`, AuthGate menampilkan LoginScreen lagi.

⚠️ **Kesalahan umum**

- Meng-import `material.dart` di provider untuk memanggil `ScaffoldMessenger` atau `Navigator`. Kembalikan hasil (`bool`/pesan) dan biarkan UI yang bereaksi.
- Mencampur AuthGate dengan `pushReplacement` manual setelah login. Pilih satu cara; jika memakai AuthGate, biarkan ia yang menentukan halaman.
- Menaruh semua class di `main.dart` sampai ratusan baris. Pecah sesuai struktur di atas sejak awal.

10. Kesalahan Umum & Cara Memperbaikinya

Bagian ini merangkum kesalahan Provider yang paling sering muncul di praktikum, lengkap dengan pesan error dan perbaikannya.

1. read di build, watch di callback

```
// ❌ SALAH
class TampilanSalah extends StatelessWidget {
  const TampilanSalah({super.key});

  @override
  Widget build(BuildContext context) {
    // read di build: angka TIDAK pernah ter-update di layar
    final nilai = context.read<CounterModel>().nilai;
    return ElevatedButton(
      // watch di callback: error "Tried to listen to a value exposed
      // with provider, from outside of the widget tree"
      onPressed: () => context.watch<CounterModel>().tambah(),
      child: Text('$nilai'),
    );
  }
}

// ✅ BENAR
class TampilanBenar extends StatelessWidget {
  const TampilanBenar({super.key});

  @override
  Widget build(BuildContext context) {
    final nilai = context.watch<CounterModel>().nilai; // watch di build
    return ElevatedButton(
      onPressed: () => context.read<CounterModel>().tambah(), // read di callback
      child: Text('$nilai'),
    );
  }
}
```

2. Memanggil provider di initState

`initState` berjalan saat widget sedang dibangun. Jika method provider langsung memanggil `notifyListeners()`, Flutter menolak karena ada widget lain yang diminta rebuild di tengah proses build.

```
@override
void initState() {
  super.initState();

  // ❌ SALAH: muatData() langsung memanggil notifyListeners() padahal
  // Flutter sedang membangun widget -> error
  // "setState() or markNeedsBuild() called during build"
  // context.read<DataProvider>().muatData();

  // ✅ BENAR: jalankan setelah frame pertama selesai digambar
```

```
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!mounted) return;
    context.read<DataProvider>().muatData();
  });

  // ✅ Setara, gaya lama: Provider.of dengan listen: false
  // WidgetsBinding.instance.addPostFrameCallback((_) {
  //   Provider.of<DataProvider>(context, listen: false).muatData();
  // });
}

// ❌ context.watch TIDAK boleh dipanggil di initState (hanya di build)
```

3. notifyListeners setelah dispose

Provider yang dibuat khusus untuk satu halaman akan di-dispose saat halaman ditutup. Jika request async masih berjalan, `notifyListeners()` di akhir request akan error.

```dart
import 'package:flutter/foundation.dart';

class PencarianProvider extends ChangeNotifier {
  bool _sudahDispose = false;
  List<String> hasil = [];

  Future<void> cari(String kata) async {
    await Future.delayed(const Duration(seconds: 3)); // request lama
    hasil = ['$kata 1', '$kata 2'];
    // Jika halaman sudah ditutup selama 3 detik itu, provider sudah di-dispose.
    // Memanggil notifyListeners() sekarang -> error
    // "A PencarianProvider was used after being disposed".
    if (_sudahDispose) return; // ✅ penjaga
    notifyListeners();
  }

  @override
  void dispose() {
    _sudahDispose = true;
    super.dispose();
  }
}
```

4. Meneruskan provider ke rute baru

```
// ✅ Meneruskan provider yang SUDAH ADA ke rute baru: pakai .value
final keranjang = context.read<KeranjangProvider>();
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => ChangeNotifierProvider.value(
      value: keranjang,
      child: const DetailPage(),
    ),
  ),
);

// ❌ SALAH: create: (_) => keranjang  -> provider akan men-dispose
```

```
//    objek milik orang lain saat DetailPage ditutup.
// ❌ SALAH: ChangeNotifierProvider.value(value: KeranjangProvider())
//    -> objek baru dibuat ulang setiap build dan tidak pernah di-dispose.
```

Contoh lengkap: semua perbaikan dalam satu aplikasi

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Contoh BENAR: memuat data di initState + berbagi provider ke halaman lain
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class PengumumanProvider extends ChangeNotifier {
  bool _sudahDispose = false;
  bool _isLoading = false;
  List<String> _items = [];
  final Set<String> _dibaca = {};

  bool get isLoading => _isLoading;
  List<String> get items => List.unmodifiable(_items);
  bool sudahDibaca(String p) => _dibaca.contains(p);

  Future<void> muat() async {
    _isLoading = true;
    _aman();
    await Future.delayed(const Duration(seconds: 1));
    _items = ['Libur Dies Natalis', 'Jadwal UTS', 'Beasiswa PPA'];
    _isLoading = false;
    _aman();
  }

  void tandaiDibaca(String p) {
    _dibaca.add(p);
    _aman();
  }

  // notifyListeners yang tidak crash walau provider sudah di-dispose
  void _aman() {
    if (!_sudahDispose) notifyListeners();
  }

  @override
  void dispose() {
    _sudahDispose = true;
    super.dispose();
  }
}

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blueGrey),
      home: Scaffold(
        appBar: AppBar(title: const Text('Menu')),
        body: Builder(
          builder: (context) => Center(
            child: FilledButton(
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute(
                  // Provider khusus halaman: dibuat saat halaman dibuka,
                  // di-dispose otomatis saat halaman ditutup.
                  builder: (_) => ChangeNotifierProvider(
                    create: (_) => PengumumanProvider(),
                    child: const PengumumanPage(),
                  ),
                ),
              ),
              child: const Text('Buka Pengumuman'),
            ),
          ),
        ),
      ),
    );
  }
}

class PengumumanPage extends StatefulWidget {
  const PengumumanPage({super.key});

  @override
  State<PengumumanPage> createState() => _PengumumanPageState();
}

class _PengumumanPageState extends State<PengumumanPage> {
  @override
  void initState() {
    super.initState();
    // Muat data SETELAH frame pertama, bukan langsung di initState
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;
      context.read<PengumumanProvider>().muat();
    });
  }

  @override
  Widget build(BuildContext context) {
    final prov = context.watch<PengumumanProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Pengumuman')),
      body: prov.isLoading
          ? const Center(child: CircularProgressIndicator())
          : ListView(
              children: [
                for (final p in prov.items)
                  ListTile(
                    title: Text(p),
```

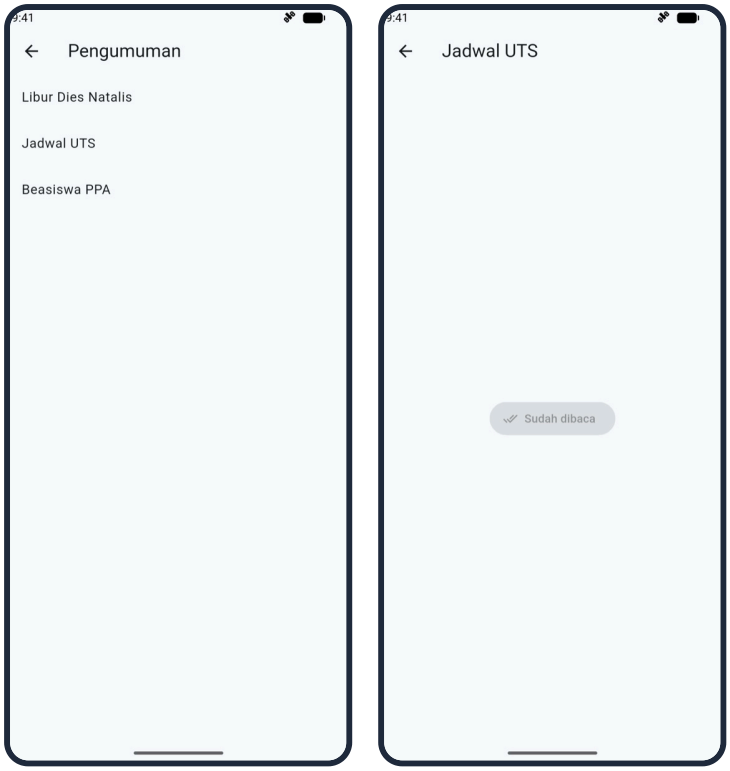
```
                    trailing: prov.sudahDibaca(p)
                        ? const Icon(Icons.done_all, color: Colors.green)
                        : null,
                    onTap: () {
                      // Bagikan provider yang SAMA ke halaman detail
                      final pengumuman = context.read<PengumumanProvider>();
                      Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (_) => ChangeNotifierProvider.value(
                            value: pengumuman,
                            child: DetailPengumumanPage(judul: p),
                          ),
                        ),
                      );
                    },
                  ),
                ],
              ),
    );
  }
}

class DetailPengumumanPage extends StatelessWidget {
  final String judul;

  const DetailPengumumanPage({super.key, required this.judul});

  @override
  Widget build(BuildContext context) {
    final dibaca = context.select((PengumumanProvider p) => p.sudahDibaca(judul));

    return Scaffold(
      appBar: AppBar(title: Text(judul)),
      body: Center(
        child: FilledButton.icon(
          icon: Icon(dibaca ? Icons.done_all : Icons.mark_email_read),
          label: Text(dibaca ? 'Sudah dibaca' : 'Tandai sudah dibaca'),
          onPressed: dibaca
              ? null
              : () => context.read<PengumumanProvider>().tandaiDibaca(judul),
        ),
      ),
    );
  }
}
```



Data dimuat setelah frame pertama

Halaman detail memakai provider yang sama (.value)

Penjelasan:

- `PengumumanProvider` dibuat di dalam `MaterialPageRoute`, sehingga hidup selama halaman Pengumuman terbuka saja.
- Data dimuat lewat `addPostFrameCallback` di `initState`, dengan cek `mounted`.
- Halaman detail menerima provider yang sama lewat `ChangeNotifierProvider.value`. Status "sudah dibaca" langsung terlihat di daftar saat kembali.
- `_aman()` mencegah error jika halaman ditutup sebelum `muat()` selesai.

Gejala / pesan error	Penyebab	Perbaikan
Angka di layar tidak berubah	Lupa `notifyListeners()` atau memakai `read` di build	Tambahkan notify; ganti dengan `watch` / `select`
Tried to listen to a value exposed with provider, from outside of the widget tree	`watch` di callback	Pakai `read`
Could not find the correct Provider<X> above this Widget	Provider tidak berada di atas widget/halaman	Pindahkan ke atas `MaterialApp` atau pakai `.value`
setState() or markNeedsBuild() called during build	Notify di `initState` / `build`	`addPostFrameCallback`
A X was used after being disposed	Notify setelah halaman ditutup	Penjaga `_sudahDispose`
Data berubah tanpa pengecekan aturan	Getter mengembalikan list asli	`List.unmodifiable`

11. Studi Kasus: Aplikasi Tugas Kuliah Multi-Halaman

Studi kasus penutup: aplikasi **Tugas Kuliah** (todo) dengan fitur tambah, centang selesai, hapus (geser), filter, dan halaman statistik. Satu `TodoProvider` dipakai bersama oleh tiga halaman.

Struktur folder

```
lib/
├── main.dart
├── models/todo.dart
├── providers/todo_provider.dart
└── screens/
    ├── todo_list_screen.dart
    ├── tambah_todo_screen.dart
    └── statistik_screen.dart
```

Langkah 1: model

`lib/models/todo.dart`

```dart
// lib/models/todo.dart
class Todo {
  final String id;
  final String judul;
  final String matkul;
  final bool selesai;

  const Todo({
    required this.id,
    required this.judul,
    required this.matkul,
    this.selesai = false,
  });

  // Membuat salinan dengan sebagian field diubah (objek tetap immutable)
  Todo copyWith({String? judul, String? matkul, bool? selesai}) {
    return Todo(
      id: id,
      judul: judul ?? this.judul,
      matkul: matkul ?? this.matkul,
      selesai: selesai ?? this.selesai,
    );
  }
}

enum FilterTodo { semua, aktif, selesai }
```

`copyWith` membuat salinan objek dengan sebagian field diganti. Objek `Todo` tidak pernah diubah langsung; yang diganti adalah elemen di list.

Langkah 2: provider

`lib/providers/todo_provider.dart`

```dart
// lib/providers/todo_provider.dart
import 'package:flutter/foundation.dart';

import '../models/todo.dart';

class TodoProvider extends ChangeNotifier {
  final List<Todo> _todos = [
    const Todo(id: '1', judul: 'Laporan praktikum Flutter', matkul: 'Pemrograman Mobile'),
    const Todo(id: '2', judul: 'Normalisasi tabel KRS', matkul: 'Basis Data', selesai: true),
  ];
  FilterTodo _filter = FilterTodo.semua;

  // ---------- GETTER (data turunan dihitung, tidak disimpan) ----------
  FilterTodo get filter => _filter;
  int get jumlahTotal => _todos.length;
  int get jumlahSelesai => _todos.where((t) => t.selesai).length;
  int get jumlahAktif => jumlahTotal - jumlahSelesai;
  double get progres => jumlahTotal == 0 ? 0 : jumlahSelesai / jumlahTotal;

  List<Todo> get todosTampil {
    switch (_filter) {
      case FilterTodo.semua:
        return List.unmodifiable(_todos);
      case FilterTodo.aktif:
        return _todos.where((t) => !t.selesai).toList();
      case FilterTodo.selesai:
        return _todos.where((t) => t.selesai).toList();
    }
  }

  Todo? cariById(String id) {
    for (final t in _todos) {
      if (t.id == id) return t;
    }
    return null;
  }

  // ---------- AKSI (mengubah state + notifyListeners) ----------
  void tambah(String judul, String matkul) {
    final id = DateTime.now().microsecondsSinceEpoch.toString();
    _todos.insert(0, Todo(id: id, judul: judul, matkul: matkul));
    notifyListeners();
  }

  void toggle(String id) {
    final i = _todos.indexWhere((t) => t.id == id);
    if (i == -1) return;
    _todos[i] = _todos[i].copyWith(selesai: !_todos[i].selesai);
    notifyListeners();
  }

  void hapus(String id) {
    _todos.removeWhere((t) => t.id == id);
    notifyListeners();
```

```
  }

  void hapusYangSelesai() {
    _todos.removeWhere((t) => t.selesai);
    notifyListeners();
  }

  void ubahFilter(FilterTodo f) {
    if (f == _filter) return;
    _filter = f;
    notifyListeners();
  }
}
```

- Data turunan (`jumlahAktif`, `progres`, `todosTampil`) dihitung dari `_todos`, tidak disimpan terpisah, sehingga tidak mungkin "tidak sinkron".
- Setiap aksi diakhiri `notifyListeners()`.

Langkah 3: main.dart

`lib/main.dart`

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/todo_provider.dart';
import 'screens/todo_list_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TodoProvider(),
      child: const TodoApp(),
    ),
  );
}

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tugas Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const TodoListScreen(),
    );
  }
}
```

Langkah 4: halaman daftar

`lib/screens/todo_list_screen.dart`

```dart
// lib/screens/todo_list_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/todo.dart';
import '../providers/todo_provider.dart';
import 'statistik_screen.dart';
import 'tambah_todo_screen.dart';

class TodoListScreen extends StatelessWidget {
  const TodoListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tugas Kuliah'),
        actions: [
          // Hanya badge ini yang rebuild saat jumlah aktif berubah
          Selector<TodoProvider, int>(
            selector: (_, p) => p.jumlahAktif,
            builder: (context, aktif, child) => IconButton(
              tooltip: 'Statistik',
              icon: Badge(label: Text('$aktif'), child: const Icon(Icons.insights)),
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute(builder: (_) => const StatistikScreen()),
              ),
            ),
          ),
        ],
      ),
      body: const Column(
        children: [
          _FilterBar(),
          Expanded(child: _DaftarTodo()),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        icon: const Icon(Icons.add),
        label: const Text('Tugas Baru'),
        onPressed: () => Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const TambahTodoScreen()),
        ),
      ),
    );
  }
}

class _FilterBar extends StatelessWidget {
  const _FilterBar();

  @override
```

```
  Widget build(BuildContext context) {
    final filter = context.select((TodoProvider p) => p.filter);

    return Padding(
      padding: const EdgeInsets.all(12),
      child: SegmentedButton<FilterTodo>(
        segments: const [
          ButtonSegment(value: FilterTodo.semua, label: Text('Semua')),
          ButtonSegment(value: FilterTodo.aktif, label: Text('Aktif')),
          ButtonSegment(value: FilterTodo.selesai, label: Text('Selesai')),
        ],
        selected: {filter},
        onSelectionChanged: (pilihan) =>
            context.read<TodoProvider>().ubahFilter(pilihan.first),
      ),
    );
  }
}

class _DaftarTodo extends StatelessWidget {
  const _DaftarTodo();

  @override
  Widget build(BuildContext context) {
    final todos = context.watch<TodoProvider>().todosTampil;

    if (todos.isEmpty) {
      return const Center(child: Text('Tidak ada tugas di filter ini'));
    }

    return ListView.builder(
      itemCount: todos.length,
      itemBuilder: (context, i) {
        final t = todos[i];
        return Dismissible(
          key: ValueKey(t.id),
          direction: DismissDirection.endToStart,
          background: Container(
            color: Colors.red,
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 20),
            child: const Icon(Icons.delete, color: Colors.white),
          ),
          onDismissed: (_) {
            context.read<TodoProvider>().hapus(t.id);
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('"${t.judul}" dihapus')),
            );
          },
          child: CheckboxListTile(
            value: t.selesai,
            onChanged: (_) => context.read<TodoProvider>().toggle(t.id),
            title: Text(
              t.judul,
              style: TextStyle(
                decoration: t.selesai ? TextDecoration.lineThrough : null,
              ),
            ),
            subtitle: Text(t.matkul),
```

```
          ),
        );
      },
    );
  }
}
```

Langkah 5: halaman tambah

`lib/screens/tambah_todo_screen.dart`

```dart
// lib/screens/tambah_todo_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/todo_provider.dart';

class TambahTodoScreen extends StatefulWidget {
  const TambahTodoScreen({super.key});

  @override
  State<TambahTodoScreen> createState() => _TambahTodoScreenState();
}

class _TambahTodoScreenState extends State<TambahTodoScreen> {
  final _formKey = GlobalKey<FormState>();
  final _judulController = TextEditingController();
  String _matkul = 'Pemrograman Mobile';

  static const daftarMatkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  @override
  void dispose() {
    _judulController.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    // Tidak perlu mengembalikan data lewat pop: provider yang menyimpan
    context.read<TodoProvider>().tambah(_judulController.text.trim(), _matkul);
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Baru')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judulController,
              decoration: const InputDecoration(
                labelText: 'Judul tugas',
                border: OutlineInputBorder(),
```

```
              ),
              validator: (v) =>
                  (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            DropdownButtonFormField<String>(
              initialValue: _matkul,
              decoration: const InputDecoration(
                labelText: 'Mata kuliah',
                border: OutlineInputBorder(),
              ),
              items: [
                for (final m in daftarMatkul) DropdownMenuItem(value: m, child: Text(m)),
              ],
              onChanged: (v) => setState(() => _matkul = v ?? _matkul),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

Langkah 6: halaman statistik

lib/screens/statistik_screen.dart

```
// lib/screens/statistik_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/todo_provider.dart';

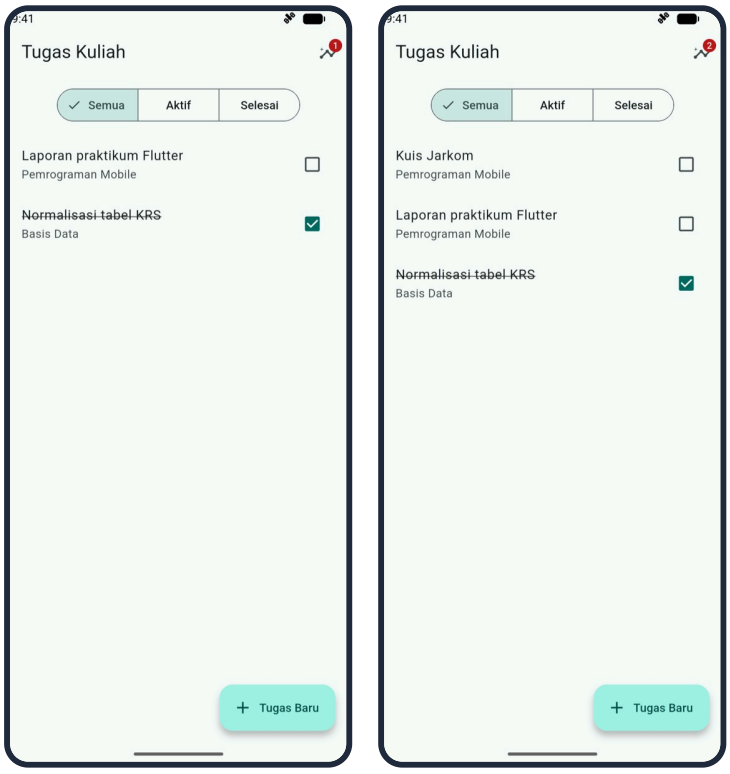
class StatistikScreen extends StatelessWidget {
  const StatistikScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final p = context.watch<TodoProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Statistik')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('${(p.progres * 100).round()}% selesai',
                style: Theme.of(context).textTheme.headlineMedium),
            const SizedBox(height: 8),
            LinearProgressIndicator(value: p.progres, minHeight: 10),
            const SizedBox(height: 16),
            Card(
              child: Column(
                children: [
                  ListTile(title: const Text('Total tugas'), trailing: Text('${p.jumlahTotal}')),
```

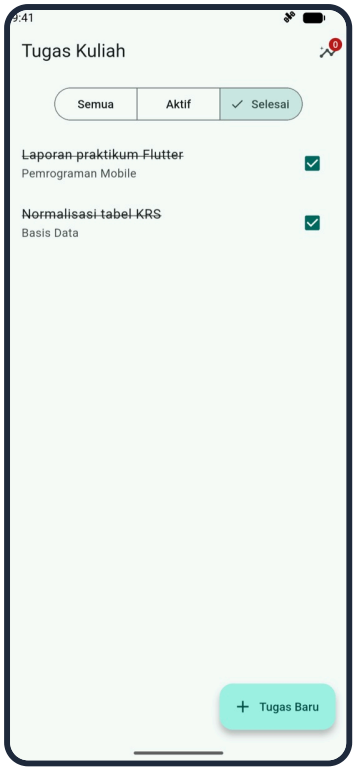
```
                ListTile(title: const Text('Aktif'), trailing: Text('${p.jumlahAktif}')),
                ListTile(title: const Text('Selesai'), trailing: Text('${p.jumlahSelesai}')),
              ],
            ),
          ),
          const SizedBox(height: 16),
          OutlinedButton.icon(
            icon: const Icon(Icons.cleaning_services),
            label: const Text('Hapus semua yang selesai'),
            onPressed: p.jumlahSelesai == 0
                ? null
                : () => context.read<TodoProvider>().hapusYangSelesai(),
          ),
        ],
      ),
    ),
  );
 }
}
```

Hasil

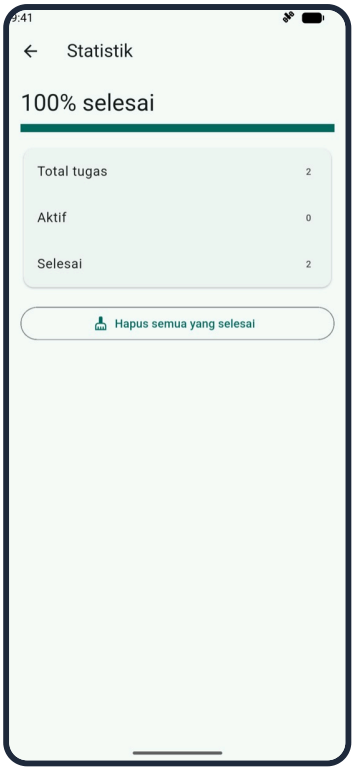


Daftar tugas awal

Tugas baru muncul, badge aktif bertambah



Filter Selesai setelah satu tugas dicentang



Halaman statistik membaca provider yang sama

Ringkasan teknik yang dipakai:

Bagian	Teknik	Alasan
Badge statistik di AppBar	`Selector<TodoProvider, int>`	Hanya ikon yang rebuild saat jumlah aktif berubah
Filter	`context.select` pada `filter`	Tombol filter tidak rebuild saat todo dicentang
Daftar todo	`context.watch`	Butuh seluruh daftar terkini
Tambah tugas	`context.read` lalu `Navigator.pop`	Tidak perlu mengembalikan data lewat pop; provider yang menyimpan
Hapus	`Dismissible` + `ValueKey(t.id)`	Key unik agar Flutter tidak salah menghapus item

ℹ️ Bandingkan dengan Modul 07

Di studi kasus Modul 07, data KRS harus dioper lewat constructor dan callback, dan halaman tambah mengembalikan hasil lewat `Navigator.pop`. Di sini, ketiga halaman cukup membaca `TodoProvider`. Pola yang sama dipakai di proyek CRUD Modul 13–14, dengan tambahan status loading/error dari section 7 dan auth gate dari section 9.

💡 Tantangan tambahan

- Tambahkan halaman edit yang memakai `cariById` dan method baru `ubah(id, judul)`.
- Tambahkan tombol "Urungkan" pada SnackBar hapus.
- Simpan todo ke penyimpanan lokal (Modul 10/11) dengan memanggil service dari provider.

Tugas Praktikum: Praktikum 8: Sistem Pengambilan SKS & Keranjang Rencana Studi

Sasaran Praktikum

Menerapkan Provider untuk sinkronisasi state keranjang KRS antar beberapa layar secara real-time, lengkap dengan pemuatan data async dan struktur folder yang rapi.

Langkah Pengerjaan

1. Tambahkan package `provider` ke dalam proyek menggunakan perintah: `flutter pub add provider`.
2. Susun folder `lib/models`, `lib/services`, `lib/providers`, `lib/screens`, dan `lib/widgets`.
3. Buat model `MataKuliah` dan `MatkulService` yang mengembalikan daftar mata kuliah setelah `Future.delayed` 2 detik.
4. Buat `KatalogProvider` dengan pola `isLoading` / `error` / `data`, lengkap dengan tampilan loading, error + tombol Coba Lagi, dan `RefreshIndicator`.
5. Buat `KrsProvider` yang mencatat mata kuliah yang dipilih, total SKS, dan saldo kuota SKS tersisa. Getter list harus mengembalikan `List.unmodifiable`.
6. Daftarkan kedua provider dengan `MultiProvider` di atas `MaterialApp`.
7. Buat Layar 1 (Katalog Matkul): Menampilkan daftar mata kuliah dengan tombol '+ Ambil Matkul'. Tombol ini memanggil `context.read<KrsProvider>().tambahMatkul(...)`.
8. Cegah penambahan jika kuota melebihi 24 SKS atau mata kuliah sudah diambil, dan tampilkan SnackBar peringatan.
9. Buat Layar 2 (Kartu Rencana Studi): Menampilkan daftar mata kuliah yang sudah diambil, tombol hapus dengan dialog konfirmasi, dan total kalkulasi SKS.
10. Pasang badge angka di ikon tab bar yang otomatis bertambah saat mata kuliah baru diambil. Gunakan `context.select` atau `Selector` agar hanya badge yang rebuild.
11. Muat data katalog dari `initState` memakai `WidgetsBinding.instance.addPostFrameCallback`.
12. Bonus: tambahkan `AuthProvider` sederhana dan `AuthGate` yang menampilkan layar login sebelum katalog.

Output yang Diharapkan

Dua atau lebih halaman yang saling terhubung secara reaktif tanpa perlu restart halaman; saat mata kuliah diambil di halaman 1, angka di halaman 2 dan badge tab langsung bertambah. Katalog menampilkan loading, error, dan data dengan benar.

Rubrik Penilaian

Desain class ChangeNotifier dan penggunaan notifyListeners (25%), Penggunaan context.watch, read, dan select yang tepat (25%), Pola async isLoading/error/data (20%), Pencegahan over-quota 24 SKS dan feedback UI (15%), Struktur folder model/service/provider/UI (15%). Bonus AuthGate +10%.

Kuis Pemahaman

1. Metode pemanggilan Provider manakah yang harus digunakan di dalam fungsi 'onPressed: () {}' sebuah tombol?

A. context.watch<T>()

B. context.read<T>()

C. setState()

D. Provider.of(listen: true)

2. Apa fungsi pemanggilan method 'notifyListeners()' di dalam class yang mewarisi ChangeNotifier?

A. Mengirim sinyal lonceng notifikasi ke server backend

B. Memberitahu framework Flutter bahwa state internal telah berubah, sehingga seluruh widget pendengar (listener) akan di-rebuild secara otomatis

C. Menutup aplikasi secara paksa

D. Menghapus cache memori smartphone

3. Sebuah widget hanya membutuhkan totalSks dari KrsProvider dan tidak ingin rebuild ketika field lain berubah. Cara paling tepat adalah ...

A. context.select((KrsProvider p) ⇒ p.totalSks)

B. context.read<KrsProvider>().totalSks

C. context.watch<KrsProvider>()

D. Membuat KrsProvider baru di widget tersebut

4. Memanggil context.read<DataProvider>().muatData() langsung di initState (dan muatData langsung memanggil notifyListeners) menyebabkan error. Perbaikannya adalah ...

A. Membungkusnya dengan WidgetsBinding.instance.addPostFrameCallback

B. Mengganti read dengan watch

C. Memindahkan panggilan ke dalam build()

D. Menghapus notifyListeners dari provider

5. Pada pola state async, mengapa _isLoading = false sebaiknya diletakkan di blok finally?

A. Agar loading selalu dimatikan, baik request berhasil maupun gagal

B. Agar request berjalan lebih cepat

C. Karena Dart melarang assignment di blok try

D. Agar error tidak pernah terjadi

6. Bagian builder mana yang di-build ulang ketika memakai Consumer<T>?

A. Hanya widget yang dikembalikan oleh builder Consumer

B. Seluruh halaman tempat Consumer berada

C. Seluruh aplikasi mulai dari MaterialApp

D. Tidak ada, Consumer tidak pernah rebuild

7. Halaman A memiliki provider. Halaman B dibuka dengan Navigator.push dan perlu memakai OBJEK provider yang sama. Cara yang benar adalah ...

A. Membungkus halaman B dengan ChangeNotifierProvider.value(value: objekYangSudahAda, ...)

B. Membungkus halaman B dengan ChangeNotifierProvider(create: (_) ⇒ objekYangSudahAda)

C. Membuat objek provider baru di halaman B

D. Mengirim provider melalui Navigator.pop

8. Dalam struktur folder model/service/provider/UI, lapisan mana yang seharusnya memanggil HTTP atau database?

A. services

B. screens

C. models

D. widgets

Kunci Jawaban & Pembahasan

1. B — context.read<T>() digunakan untuk memicu aksi/method tanpa mendaftarkan widget untuk rebuild saat terjadi perubahan nilai. Memanggil context.watch di dalam callback tombol dilarang oleh Flutter karena tombol tidak berada di alur build.

2. B — notifyListeners() adalah jantung dari ChangeNotifier yang memancarkan sinyal ke semua Consumer atau context.watch yang sedang mengamati model tersebut untuk memperbarui tampilannya.

3. A — select hanya memicu rebuild jika nilai yang dipilih berubah. read di build tidak pernah memicu rebuild, sedangkan watch memicu rebuild untuk setiap notifyListeners.

4. A — notifyListeners saat widget sedang dibangun menimbulkan error 'setState() or markNeedsBuild() called during build'. addPostFrameCallback menjalankan kode setelah frame pertama selesai.

5. A — Blok finally selalu dijalankan setelah try/catch. Tanpa finally, jika terjadi exception, isLoading bisa tetap true dan UI terjebak di tampilan loading.

6. A — Consumer membatasi rebuild pada builder-nya saja. Widget di luar Consumer (induknya) tidak ikut di-build ulang.

7. A — .value dipakai untuk objek yang sudah ada dan tidak akan men-dispose objek tersebut. Memakai create dengan objek yang sudah ada menyebabkan objek di-dispose saat halaman B ditutup.

8. A — Service adalah satu-satunya lapisan yang tahu cara mengakses data. Provider memanggil service, UI memanggil provider. Dengan begitu sumber data mudah diganti dan diuji.